

MGPEB — Módulo de Gerenciamento de Pouso e Estabilização de Base

Relatório técnico — Aurora Siger

Atividade Integradora — Fase 2 Equipe: preencher os nomes dos integrantes **Versão do protótipo:** 1.0 — outubro de 2026

Resumo

Este relatório apresenta um protótipo didático para organizar a autorização de pouso de módulos destinados à base Aurora Siger, em Marte. O programa cadastra módulos, recebe chegadas e eventos, verifica condições mínimas, organiza candidatos por urgência e dependência, estima consumo de combustível e registra decisões. A implementação usa listas e laços explícitos para praticar os conteúdos da disciplina.

O projeto não é um simulador aeroespacial validado. Seus coeficientes e cenários são hipóteses didáticas. Em particular, o código atual não calcula altitude, velocidade, trajetória, arrasto, temperatura ou frenagem real. Essa delimitação evita confundir uma regra computacional demonstrativa com uma previsão de engenharia.

Premissas do cenário

Adota-se que uma nave de transporte permanece em órbita de Marte e libera módulos autônomos para pouso individual. Cada módulo possui massa, combustível de descida, sensores, sistemas e horário estimado de chegada próprios. A missão acompanha a ordem de chegada e autoriza no máximo um pouso por vez na área representada pelo protótipo. Esta arquitetura é uma escolha da equipe para preencher uma ambiguidade do enunciado, não uma exigência expressa nele.

A fila considera cinco tipos de módulo: Energia, Habitação, Logística, Médico e Laboratório. A ordem padrão é definida pelas dependências da base. Módulos pousados não são automaticamente considerados operacionais: o de Energia precisa estar funcional para ativar os demais; o Laboratório também aguarda Energia e Habitação.

1. Cenário e objetivo do sistema

O MGPEB representa a camada de decisão entre a chegada de um módulo à órbita e sua autorização para iniciar uma descida. A pergunta central é: **“Este módulo pode pousar agora, e, entre os autorizados, qual deve ser o próximo?”** São duas decisões diferentes. Segurança determina quem está apto; prioridade determina a ordem apenas entre os aptos.

Os módulos têm identificador, tipo, prioridade, combustível em quilogramas, massa em quilogramas, criticidade da carga, ETA em minutos, estado de sensores, estado de sistemas, resultado de acidente e campos de estado/motivo. O ETA é contado a partir do começo da simulação. Os dados são exemplos para demonstrar o algoritmo e não descrevem hardware real.

ID	Módulo	Prioridade	Combustível (kg)	Massa (kg)	Criticidade	ETA (min)
E	Energia	1	30	1.000	1	0
H	Habitação	1	30	1.000	1	0
G	Logística	1	30	1.000	1	0
M	Médico	1	30	1.000	1	0
L	Laboratório	1	30	1.000	1	0

Os valores iguais formam o cenário padrão para observar a ordem por tipo sem diferenças iniciais de massa, combustível ou criticidade. São parâmetros didáticos; os exemplos alteram dados para demonstrar outras situações. Energia sustenta a ativação dos demais módulos, e o Laboratório também depende da Habitação.

O ambiente é compartilhado: o protótipo guarda se a atmosfera está aceitável e se a área está livre, ocupada ou obstruída. Eventos programados podem mudar clima, área, sensores ou sistemas. Como simplificação, uma descida inteira é tratada como uma transição: alterações previstas durante ela são aplicadas depois que termina.

Objetivo

O programa deve demonstrar cadastro, validação de entradas, busca sequencial, ordenação por inserção, fila FIFO, pilha de eventos, condições booleanas e uma função matemática simples. O fluxo é repetível: os dados do cadastro são copiados antes de simular, então uma rodada não consome os valores da próxima.

A versão principal está em `mgpeb.py`. `exemplos.py` contém cenários de teste; `test_mgpeb.py` é uma bateria externa de validação. `main.py` é um experimento anterior de descida física e não participa da lógica atual da fila.

2. Estruturas de dados e fluxo

Cada módulo é uma lista de 12 posições. Constantes como ID, MASSA e COMBUSTIVEL dão nomes a essas posições. A escolha imita vetores apresentados em aula e deixa visível como cada campo é acessado pelo índice. A função `validar` confere quantidade e tipo dos campos, faixas numéricas, tipos aceitos e identificadores únicos antes da simulação.

A busca é linear: buscar percorre os módulos até encontrar um tipo ou identificador; `buscar_extremo` conserva o índice do menor ou maior valor encontrado. Para cinco módulos, a busca linear é suficiente e simples de explicar. Em uma lista muito maior, seria possível avaliar outras estruturas, mas isso está fora do objetivo didático atual.

A fila `fila` recebe os módulos quando o ETA chega. O primeiro índice é retirado e transferido para `espera`, que conserva os candidatos entre ciclos de decisão. A nova lista `pousados` guarda índices dos módulos que concluíram a descida com sucesso; cada índice aponta para o cadastro original em dados, sem duplicar registros. `alertas` guarda o identificador e o motivo de cada bloqueio. A área é reservada antes do pouso. Um pouso bem-sucedido a libera; um cenário de acidente a deixa obstruída.

A lista `aptos` reúne somente módulos aprovados. `ordenar` implementa ordenação por inserção: pega um candidato e desloca os anteriores até encontrar sua posição. O cadastro original não é reordenado. Empates completos mantêm a ordem de entrada, o que torna o resultado previsível.

O histórico também é copiado para `pilha`. Seu topo é o registro mais recente, seguindo LIFO. `ultimo_evento` consulta esse topo; `desfazer_consulta` devolve uma nova pilha sem ele. Essa operação demonstra a pilha, mas não desfaz pousos nem altera combustível.

Passos de cada ciclo

1. Aplicar eventos vencidos, na ordem do horário.
2. Inserir na fila os módulos cujo ETA chegou.
3. Mover as chegadas para a espera e atualizar dependências operacionais.
4. Verificar combustível, sensores, sistemas, atmosfera e área.
5. Se não houver candidatos, avançar o relógio ao próximo ETA ou evento; sem eventos futuros, encerrar com pendências.
6. Ordenar os autorizados, reservar a área, simular a duração e o consumo estimados e registrar o resultado.

3. Autorização e prioridade

A regra de segurança é uma conjunção: todas as condições obrigatórias precisam ser verdadeiras. Um motivo vazio em autorizar significa que o módulo foi aprovado. Quando alguma condição falha, o programa concatena os motivos para que a equipe saiba por que o módulo permanece aguardando.

C = combustível suficiente para descida e reserva
S = sensores operacionais
E = sistemas operacionais
A = atmosfera aceitável
D = área livre

AUTORIZAR = C AND S AND E AND A AND D

Representação da regra:

```
[C] --\\  
[S] ---\\  
[E] ---- AND ----> [Autorização]  
[A] ---/  
[D] --/
```

O protótipo ainda não verifica ângulo de entrada, coordenadas, integridade estrutural ou condição independente do paraquedas. Esses itens aparecem nas anotações do projeto e podem ser incluídos como novos sinais booleanos, com validação e cenários de teste próprios.

Como a fila escolhe o próximo

A urgência é calculada somente depois da autorização; combustível crítico nunca libera uma descida insegura. Primeiro, calcula-se a margem relativa em relação ao mínimo seguro. Uma margem de até 25% é classificada como urgente. Entre urgentes, vence a menor margem; se empatar, vence a maior criticidade. A ordem padrão por tipo e, por fim, a prioridade numérica desempata os restantes. Sem urgência, usa-se a ordem padrão e depois a prioridade.

O código usa números de prioridade de 1 a 5, em que um número maior significa maior importância, conforme a convenção deste protótipo. É importante manter essa convenção igual no relatório, no cadastro e no diagrama de decisão.

Os cenários cobrem operação normal, combustível insuficiente, sensores falhos, clima adverso, área ocupada, urgência, ETA futuro, dependência de ativação, mudança de clima e acidente. O teste automatizado confirma que bloqueios não autorizam pouso e que um acidente obstrui a área.

4. Modelo matemático: consumo de combustível

A função matemática escolhida estima o combustível necessário para uma descida. Ela é deliberadamente simples e compatível com o conteúdo de funções lineares e afins da disciplina. A taxa é uma hipótese do protótipo, não um valor publicado para um veículo marciano.

$$C(t) = k * m * t$$

$$F(t) = C(t) + R$$

Onde:

- taxa é a taxa hipotética de consumo em kg/(kg·min);
- massa é a massa do módulo em kg;
- duração é o tempo estimado de descida em minutos;
- reserva é uma quantidade fixa de combustível em kg.

Os símbolos $C(t)$ e $F(t)$ representam, respectivamente, o consumo estimado e o mínimo com reserva. k é a taxa hipotética; m é a massa; t é a duração; e R é a reserva. A unidade de $k * m * t$ resulta em quilogramas.

Parâmetro	Valor usado	Unidade/significado
k	0,002	kg/(kg·min), taxa hipotética
m	1.000	kg, massa do módulo padrão
t	5	min, duração estimada
R	2	kg, reserva fixa

Com os parâmetros do programa (taxa = 0,002, duração = 5 min, reserva = 2 kg) e um módulo de 1.000 kg, o consumo estimado é $0,002 \times 1.000 \times 5 = 10$ kg. O mínimo seguro fica em $10 + 2 = 12$ kg. Um módulo com menos de 12 kg é bloqueado; a reserva não é consumida na simulação.

Para massa e taxa fixas, o consumo cresce linearmente com a duração. No gráfico de consumo por duração, a reta parte de zero e sua inclinação é taxa $\times$ massa. Para o mínimo seguro, a reta tem a mesma inclinação, mas começa no valor da reserva. Aumentar massa ou duração aumenta o consumo e o mínimo; aumentar a reserva desloca para cima a reta do mínimo sem alterar sua inclinação.

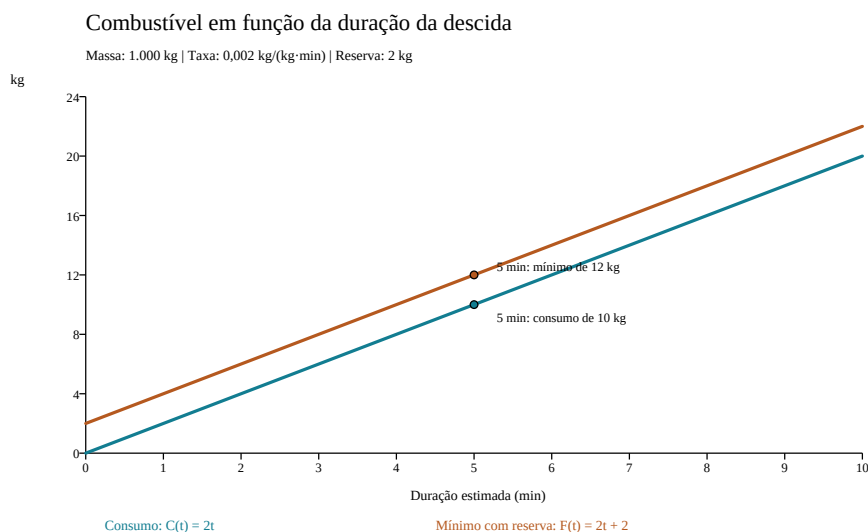


Figura 1. Consumo estimado e mínimo com reserva para massa de 1.000 kg. Elaborado a partir da fórmula e dos parâmetros do protótipo.

Duração (min)	Consumo C(t) (kg)	Mínimo F(t) (kg)
0	0	2
2	4	6
5	10	12
10	20	22

Na configuração padrão, a urgência usa a margem $(\text{combustível disponível} - F(5)) / F(5)$. Como $F(5) = 12$ kg, a faixa urgente é de 12 a 15 kg, inclusive: abaixo de 12 kg o módulo é bloqueado; até 25% acima do mínimo ele pode ser priorizado entre os autorizados.

Essa função ajuda a comparar combustível disponível com uma necessidade estimada e a justificar a fila por margem relativa, em vez de comparar apenas quilogramas brutos entre módulos diferentes. Ela não modela empuxo, gravidade, arrasto, altitude ou velocidade; portanto, não permite afirmar que o módulo tocará o solo a uma velocidade específica. Os parâmetros devem ser apresentados como hipótese didática.

5. Evolução da computação e sistemas embarcados

Os primeiros computadores eletrônicos de grande porte, como o ENIAC, foram construídos para executar cálculos em alta velocidade. Ainda ocupavam salas, usavam muitos componentes e exigiam configuração e manutenção. A evolução de componentes, memória e circuitos integrados tornou possível levar processamento para dentro de máquinas e veículos, em vez de depender apenas de um computador central.

O Apollo Guidance Computer é um marco dessa mudança: um computador embarcado participou da navegação e do controle da missão Apollo. O ponto relevante para o MGPEB não é comparar diretamente o nosso programa com software de voo real, mas perceber a mudança de ideia: o veículo precisa interpretar sinais e tomar decisões quando não há um operador fazendo cada cálculo manualmente. O Apollo Lunar Surface Journal preserva transcrições e documentação operacional que ajudam a estudar essa história.

Rovers atuais mostram essa integração com mais clareza. No Perseverance, computadores, sensores, câmeras, controle térmico, alimentação elétrica e telecomunicações trabalham juntos. A navegação relativa ao terreno analisa imagens durante a descida para selecionar uma região mais segura. Isso ilustra por que decisões de pouso precisam ser locais e rápidas: a comunicação com a Terra não substitui o controle embarcado em tempo real.

O MGPEB traduz essa ideia para os conteúdos da disciplina em escala reduzida. A fila organiza a missão; as condições booleanas representam regras; o laço avança o relógio; eventos atualizam o ambiente; e o histórico permite explicar por que cada decisão ocorreu. Assim, o programa serve como modelo conceitual, não como software de controle de voo.

6. Limitações de hardware e impacto técnico

Um computador embarcado de missão precisa operar com limites de massa, potência, memória, dissipação de calor e comunicação. Também precisa ser projetado para o ambiente de radiação. A página da NASA sobre o Perseverance informa o uso de processador RAD750 tolerante à radiação, redundância de computadores e memórias de diferentes tipos e capacidades. Ela também descreve monitoramento de energia e temperatura e controle térmico do corpo do rover.

Esses números são características daquele rover, não uma especificação universal para toda missão a Marte. O exemplo mostra, porém, por que sistemas embarcados priorizam previsibilidade, monitoramento de saúde, recuperação diante de falhas e armazenamento de telemetria. A equipe deve evitar afirmar que todo computador espacial tem as mesmas capacidades.

No protótipo, a lista de módulos é pequena e os algoritmos são simples. Busca linear custa tempo proporcional à quantidade de módulos; ordenação por inserção pode exigir muitas comparações quando a lista cresce. Para cinco módulos, a clareza vale mais do que otimização. A simulação também evita dependências externas no programa do aluno e usa estruturas que podem ser inspecionadas manualmente.

A validação e os testes automatizados são parte da confiabilidade: testar cenários normais e adversos mostra se a lógica respeita suas regras. A bateria de testes usa recursos adicionais de Python, mas está separada do código didático. Isso evita introduzir ferramentas avançadas na implementação principal e ainda fornece uma checagem repetível para a equipe.

A rede e a comunicação também são um limite operacional: o módulo precisa continuar decidindo quando a Terra não pode responder imediatamente. Por isso, bloqueios, eventos, motivos e estados são guardados localmente no protótipo.

7. ESG, segurança e governança

Ambiental: a exploração deve considerar uso de energia, combustível, resíduos e contaminação biológica. A NASA define proteção planetária como a proteção de outros corpos celestes contra contaminação terrestre e a proteção da Terra contra possível contaminação de amostras retornadas. No cenário, isso pode ser discutido como requisito de planejamento e operação; o código atual não calcula esterilização nem impacto ambiental.

Social: a ordem de pouso deve considerar dependências e consequências para a segurança da futura tripulação. Energia e Habitação sustentam funções essenciais da base. Regras claras ajudam a evitar que uma escolha de prioridade deixe de atender uma condição mínima de segurança. A criticidade da carga descreve a consequência de uma perda; prioridade descreve a sequência necessária. Não são a mesma métrica.

Governança: toda decisão precisa ter critérios verificáveis, motivo de bloqueio e histórico. O programa guarda os eventos e suas razões, valida dados antes de simular e testa casos adversos. Em um sistema real seriam necessários processos formais de engenharia, revisão independente, segurança de software, controle de versões, rastreabilidade de requisitos e planos de contingência. A NASA descreve garantia e segurança de software como atividades ao longo do ciclo de vida, incluindo avaliação independente para sistemas críticos. Os testes deste trabalho são educacionais e não equivalem a certificação.

Uma regra operacional de governança para a equipe é separar três categorias: apto, aguardando condição recuperável e alerta impeditivo. Também se deve registrar a origem de cada parâmetro: dado de fonte, hipótese de projeto ou valor apenas didático. Essa separação torna o relatório honesto e permite que outro integrante reproduza os resultados.

8. Conclusão, limites e referências

O protótipo demonstra a organização de módulos, a fila de chegadas, buscas, ordenação por inserção, regras AND, eventos, dependências e pilha de histórico. A função afim de consumo conecta um fenômeno operacional a uma decisão de autorização. Os testes verificam os cenários previstos e ajudam a detectar regressões.

O principal limite é físico: não se calculam a trajetória ou a velocidade real de pouso. Os parâmetros de consumo, duração, margem de urgência e ordem dos módulos são hipóteses da equipe. Antes de ampliar o modelo, a equipe deve escolher quais dados deseja representar, definir unidades, buscar valores justificáveis e adicionar testes para cada regra nova. Uma próxima evolução possível é incluir um modelo de temperatura interna com potência térmica e capacidade térmica, mantendo-o separado da autorização de pouso.

Referências

1. COMPUTER HISTORY MUSEUM. **ENIAC**, exposição Birth of the Computer. Disponível em: <https://www.computerhistory.org/revolution/birth-of-the-computer/4/78/> . Acesso em: 4 out. 2026.
2. MINDELL, David A. **Digital Apollo: Human and Machine in Spaceflight**. Cambridge: MIT Press, 2008.
3. NASA. **Apollo Lunar Surface Journal and Apollo Flight Journal**. Disponível em: <https://www.nasa.gov/history/alsj/> . Acesso em: 4 out. 2026.
4. NASA. **Mars 2020 Perseverance Rover Components**. Informações sobre computador, redundância, potência, controle térmico e navegação. Disponível em: <https://science.nasa.gov/mission/mars-2020-perseverance/rover-components/> . Acesso em: 4 out. 2026.
5. NASA Office of Safety and Mission Assurance. **Software Assurance and Software Safety**. Disponível em: <https://sma.nasa.gov/sma-disciplines/software-assurance-and-software-safety> . Acesso em: 4 out. 2026.
6. NASA Office of Safety and Mission Assurance. **Planetary Protection**. Disponível em: <https://sma.nasa.gov/sma-disciplines/planetary-protection> . Acesso em: 4 out. 2026.

Observação: o conteúdo histórico e técnico foi resumido com palavras próprias. Valores do simulador não são dados oficiais de desempenho de uma nave real.